Jogo da Forca em Rede

Aplicação cliente-servidor com sockets TCP

Marcos Samuel Cornelio Barros · 2025027554

Bruno Vinicius dos Reis Souza · 2025034595

Fundamentos de Redes de Computadores

PROBLEMA

Como processos separados cooperam pela rede?

Estado no servidor

O servidor sorteia a palavra e guarda letras e erros de cada jogador.

Cliente simples

O cliente só envia comandos e mostra o resultado. A palavra secreta nunca é enviada durante a rodada.

Vários jogadores

Cada cliente tem sua rodada independente, ao mesmo tempo.

SOLUÇÃO

Forca cliente-servidor sobre TCP

Servidor

Sorteia a palavra, valida mensagens, conta erros e responde.

Cliente

Envia letras pelo terminal e exibe a palavra mascarada e os erros.

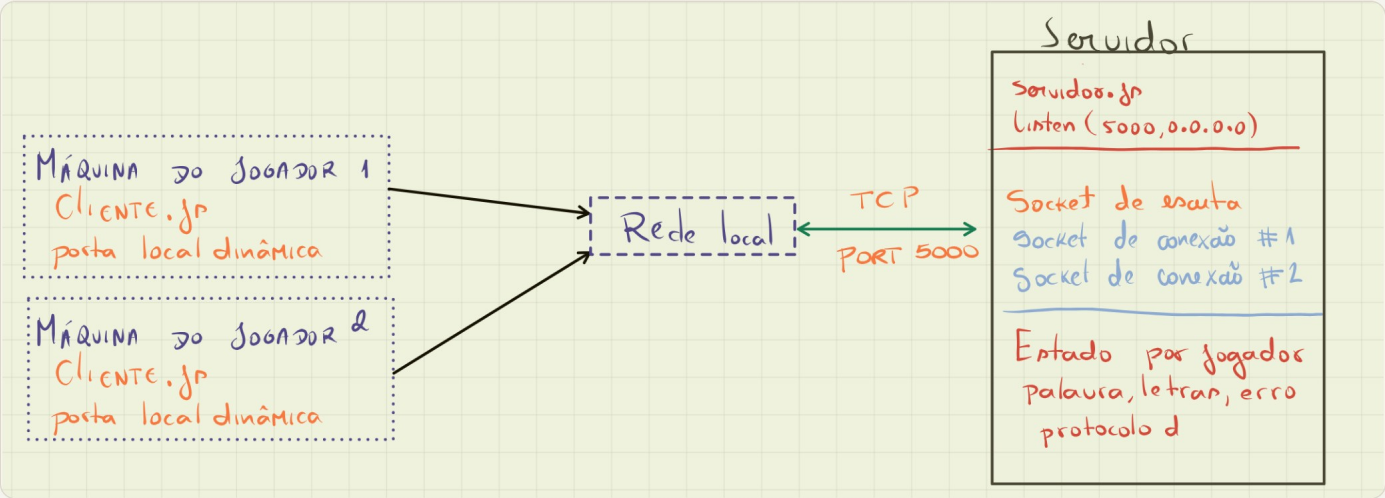
Regras

Máximo de 6 erros. A palavra só aparece em VITORIA ou DERROTA.

Exemplo de resposta do servidor `ESTADO _E_E 1 6`

ARQUITETURA

Um servidor, vários clientes



Escuta
listen na porta 5000, em 0.0.0.0

Um socket por cliente
conexão persistente e estado próprio

Concorrência
laço de eventos do Node.js

ESCOLHA DO TRANSPORTE

TCP em vez de UDP

Por que TCP

- **Confiabilidade:** ACK e retransmissão garantem cada LETRA e cada resposta
- **Ordem:** letras e respostas chegam na sequência
- **Conexão persistente:** estado por jogador e detecção de queda
- **Sem tempo real:** a latência extra não é perceptível

E se fosse UDP?

- **Perda de LETRA:** o jogador espera uma resposta que não vem
- **Perda de ESTADO:** tela desatualizada
- **Reordenação e duplicação:** máscara inconsistente, letra processada duas vezes
- **Sem conexão:** heartbeats e timeouts na aplicação

PROTOCOLO DA APLICAÇÃO

Texto simples, uma mensagem por linha

Formato COMANDO arg1 arg2, terminado em quebra de linha. O TCP é fluxo de bytes, então um buffer só entrega a mensagem ao achar a quebra de linha.

Mensagem	Sentido	Significado
ENTRAR nome	Cliente → Servidor	Identifica o jogador e inicia a rodada
LETRA x	Cliente → Servidor	Tenta uma letra de A a Z
SAIR	Cliente → Servidor	Encerra a conexão
OK texto	Servidor → Cliente	Confirmação
ESTADO máscara erros max	Servidor → Cliente	Situação atual da rodada
VITORIA / DERROTA palavra	Servidor → Cliente	Fim da rodada, nova começa
ERRO código texto	Servidor → Cliente	Mensagem inválida ou não permitida

PROTOCOLO DA APLICAÇÃO

Conversa de exemplo e erros

```
C → ENTRAR Dolinho
S → ESTADO ____ 0 6
C → LETRA e
S → ESTADO _E_E 0 6
C → LETRA z
S → ESTADO _E_E 1 6
C → LETRA e
S → ERRO 104 Letra ja tentada
C → SAIR
S → OK Ate logo
```

Código	Significado
100	Comando desconhecido
101	Argumentos inválidos
102	Jogador já autenticado
103	Ação não permitida ou inatividade
104	Letra já tentada, sem contar erro

DEMONSTRAÇÃO

Funcionando ao vivo

Terminal 1 · servidor

```
node server.js
```

Terminal 2 e 3 · clientes

```
node cliente.js
```

Outra máquina

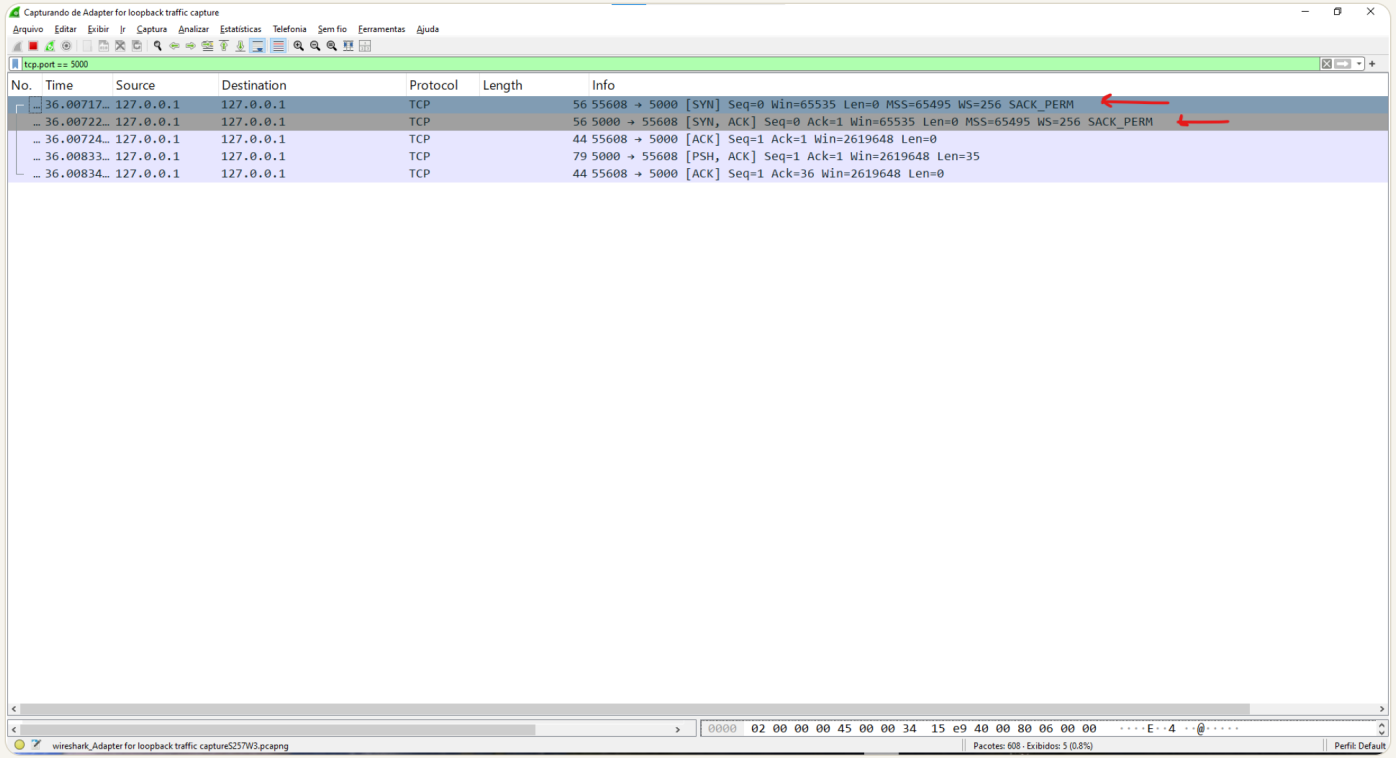
```
node cliente.js 192.168.0.10 5000
```

Roteiro

1. ENTRAR com um nome
2. Letra certa e letra errada
3. Letra repetida: ERRO 104
4. Comando inválido: ERRO 100
5. Dois clientes ao mesmo tempo
6. SAIR e inatividade de 60 s

WIRESHARK · FILTRO TCP.PORT == 5000

Handshake de 3 vias

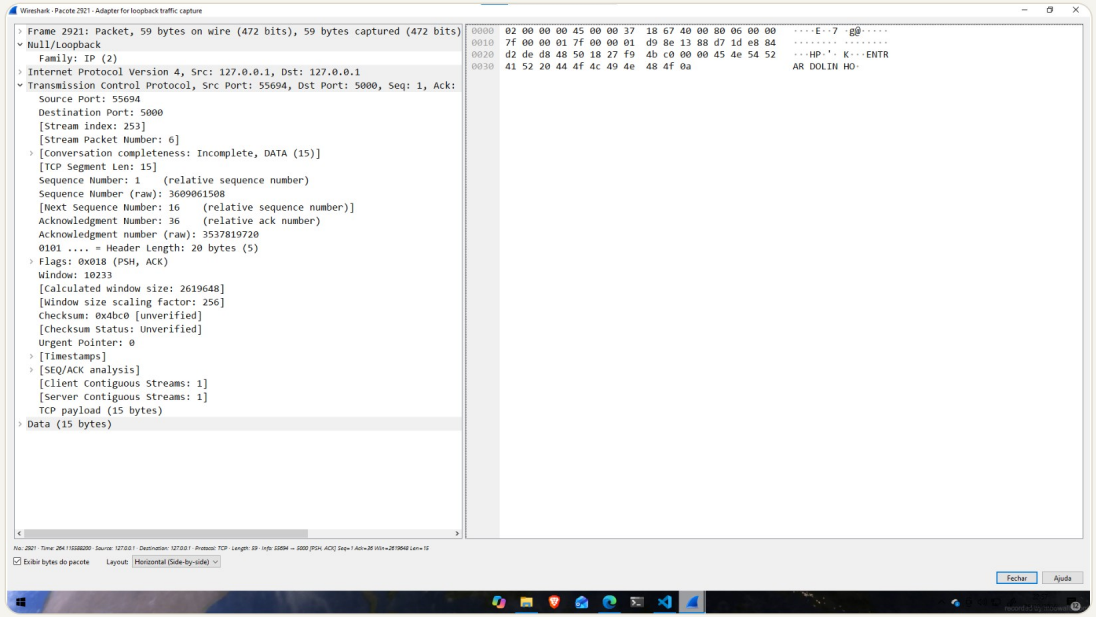


1 · SYN
Cliente (55608) para o servidor (5000)

2 · SYN, ACK
Servidor aceita a conexão

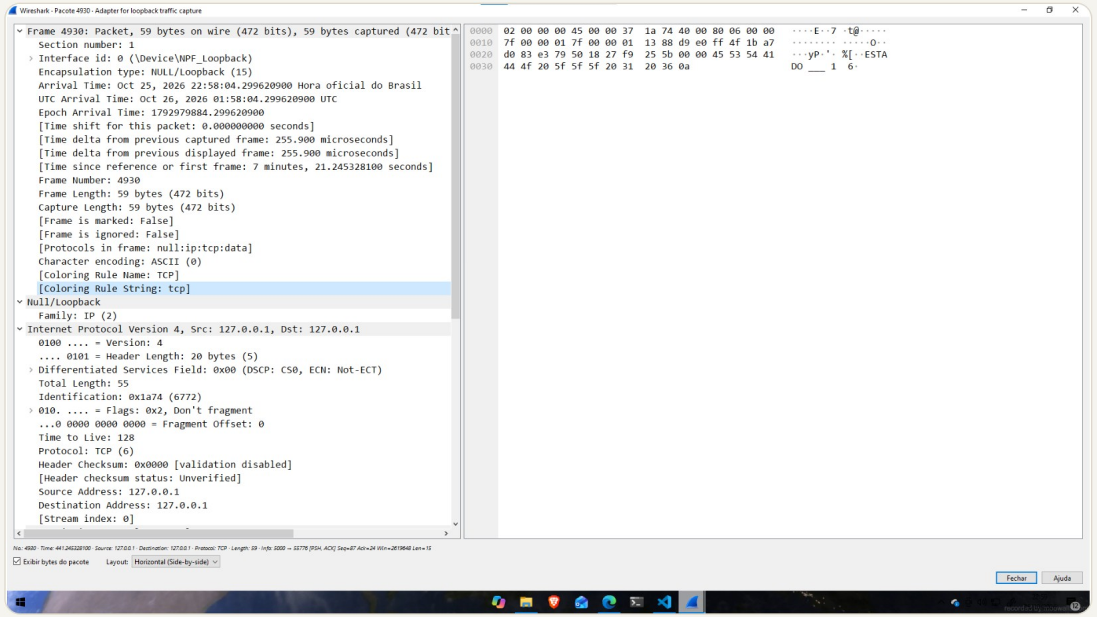
3 · ACK
Conexão estabelecida. Em seguida, o servidor envia o OK de boas-vindas (PSH, ACK, 35 bytes)

Protocolo em texto claro



`ENTRAR Dolinho`

Cliente → Servidor, PSH, ACK, 15 bytes de payload

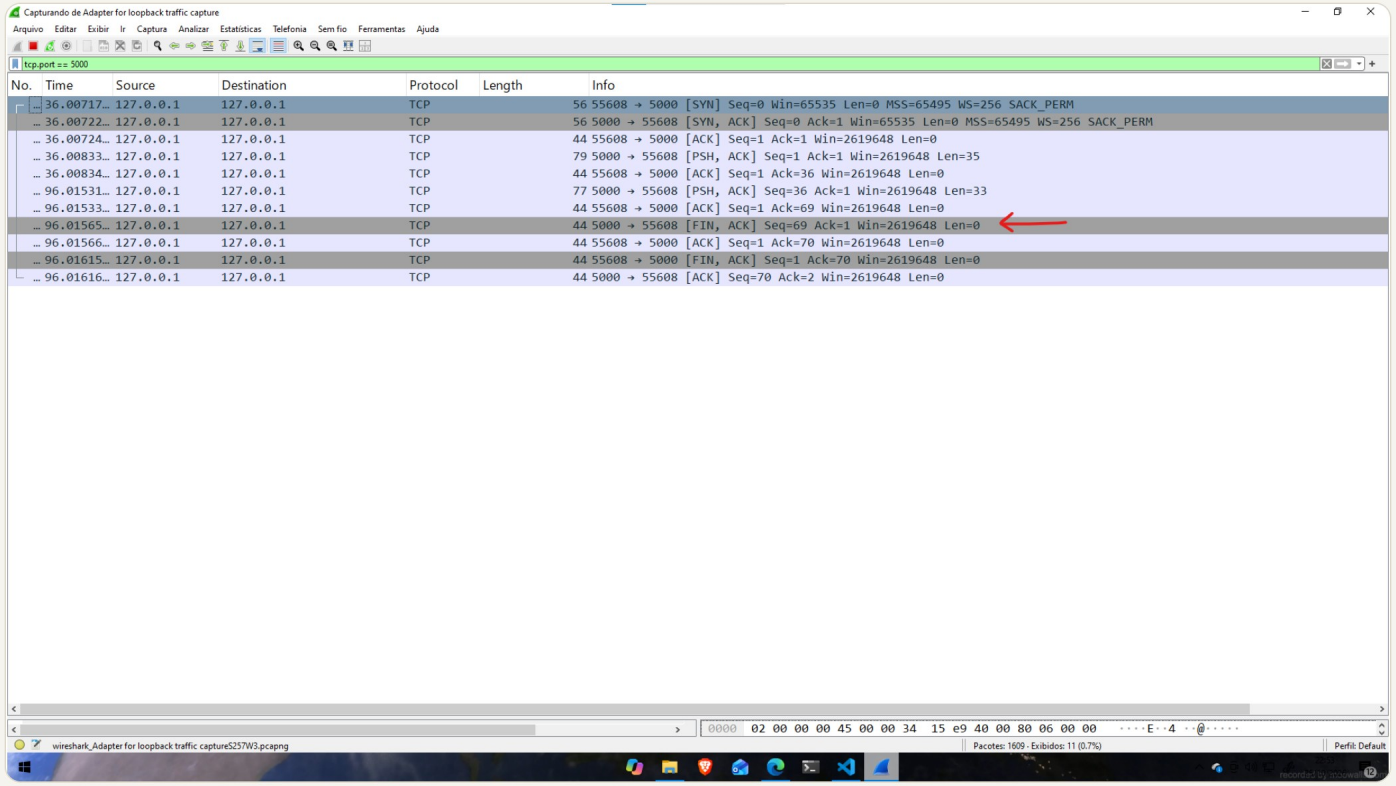


`ESTADO ___ 1 6`

Servidor → Cliente, após uma letra errada: erros sobem de 0 para 1

WIRESHARK · INATIVIDADE

Timeout de 60 s e encerramento



36 s → 96 s

60 s sem nenhum pacote da aplicação

ERRO 103

Inatividade, encerrando (33 bytes)

FIN, ACK

O servidor fecha primeiro; o cliente responde com FIN, ACK e o servidor com o ACK final

CONCLUSÃO

TCP resolveu o problema sem trabalho extra

O que funciona

- Vários jogadores simultâneos, cada um com seu estado
- Protocolo textual com delimitador e códigos de erro
- Servidor resiliente a falhas de clientes
- Tráfego comprovado no Wireshark

Limitações

- Sem criptografia: tudo em texto claro
- Sem autenticação nem pontuação persistente
- Sem limite de conexões simultâneas